

Práctica 3.3: Programación de paralela en Python

Índice

Objetivos	1
Introducción a oneAPI	1
Instalación y setup de Intel Distribution for Python	3
Ejercicios	4
Ejercicio 1: Vectorización en numpy	4
Uso de paralelismo	4
Paquetes extra	5
Ejercicio 2: Uso de numpy	5
Ejercicio 3: Uso de DPNP y el Data Parallel Extension en Python	5
Extensión de DataParallel para Numpy - dpnp	7
Contenido del laboratorio	7
Ejercicio 4	8
Ejemplo inicial	8
Ejemplo 1 (cálculo de π con el método de Montecarlo)	8
Ejemplo 2 (Kmeans)	8
Resultados esperado	10

Objetivos

- Comprender conceptos básicos de paralelismo en Python y relacionarlo con la librería Numpy
- Explotación de aceleradores tipo GPU con Python
- Uso de bibliotecas como Numpy, DPNP para realizar operaciones vectoriales y paralelas

Introducción a oneAPI

oneAPI es una es una plataforma de desarrollo abierta y unificada creada por Intel, diseñada para permitir que los desarrolladores escriban código una sola vez y lo ejecuten eficientemente en diversos tipos de hardware, como: CPUs (procesadores tradicionales), GPUs (tarjetas gráficas), FPGAs (chips reconfigurables) u otro tipo de aceleradores.

Incluye desde compiladores (ya usados en la asignatura como el Intel® oneAPI DPC++/C++ Compiler), librerías optimizadas (oneMKL, oneDNN, oneTBB...), que soportan modelos de programación de HPC y herramientas de perfilado como Intel Advisor o VTune.

El conjunto herramientas de desarrollo permiten trabajar desde CPU a XPU, y están disponibles en varios Toolkits para su uso.

Para esta práctica nosotros nos centraremos en la AI Tools que incorpora:

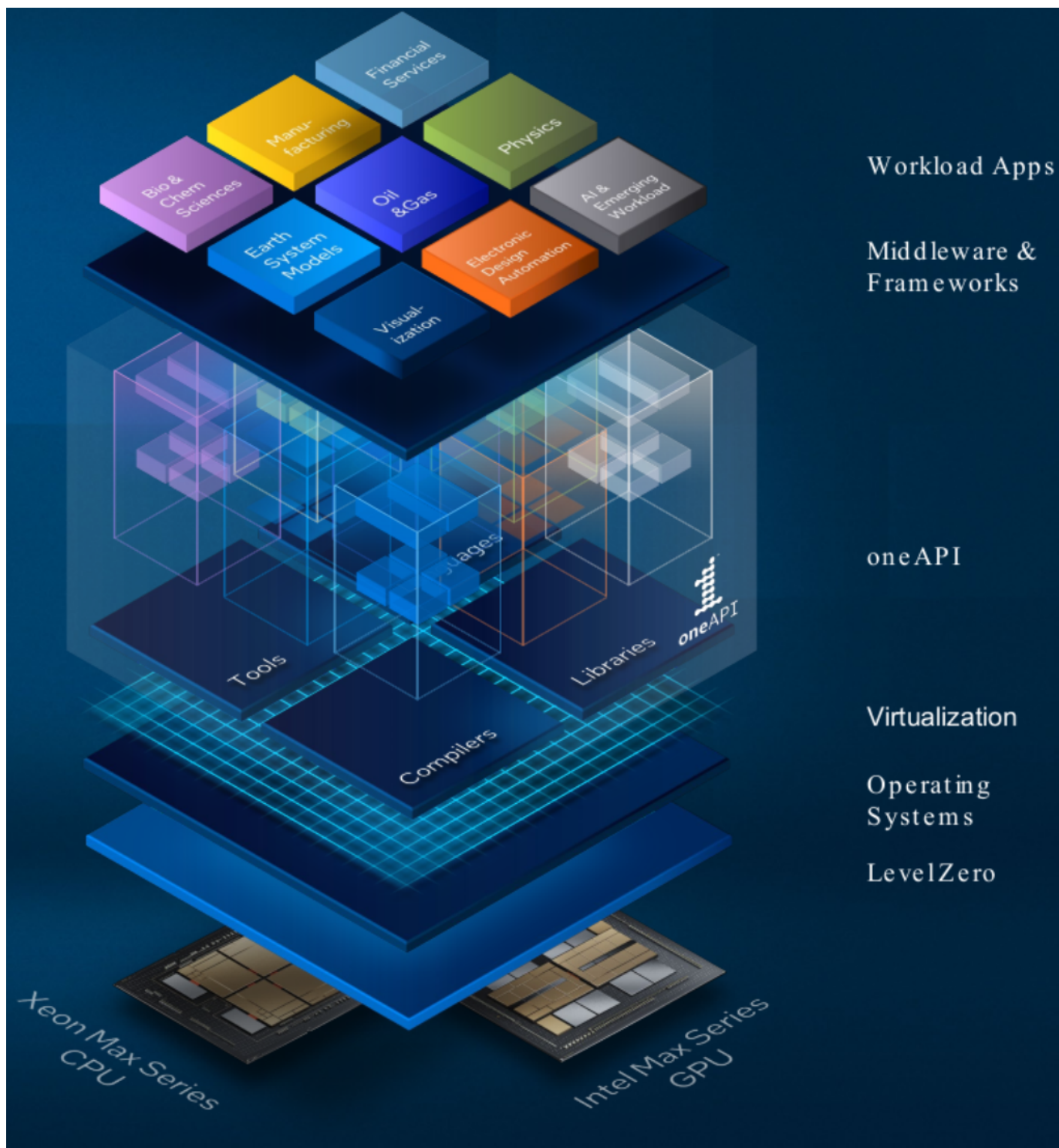


Figure 1: Fig: Modelo de programación unificado

intel AI TOOLS 1 oneAPI	Data Analytics at Scale:	Modin*	pandas*	NumPy*	SciPy*			
	DL Inference and Training:	TensorFlow*	PyTorch*	OpenVINO™	Intel® Neural Compressor			
	Classical ML:	Scikit-learn*	XGBoost*	Python*				
intel HPC TOOLKIT 1 oneAPI	Intel® Fortran CompilerIntel® MPI LibraryIntel® SHMEM Library							
intel 1 oneAPI BASE TOOLKIT	Tools	Intel® DPC++/C++ Compatibility Tool	Intel® VTune™ Profiler	Intel® Advisor	Intel® Distribution for GDB	Intel® Distribution for Python		
	Performance Libraries:	oneMKL	oneDNN	oneDAL	oneCCL	oneTBB	oneDPL	Intel® IPP
	Direct Programming:	C++ with SYCL*		C++	Python*	OpenMP*		
	Compilers:	Intel® oneAPI DPC++/C++ Compiler						
	Hardware Interface – oneAPI Level Zero & OpenCL*							

Figure 2: Fig: Toolkits y herramientas disponibles en oneAPI

- Intel® Distribution for Python: rendimiento cercano al del código nativo con este conjunto de paquetes esenciales
- Intel® Extension for PyTorch (CPU & GPU): extensión para PyTorch
- Intel® Extension for TensorFlow* (CPU & GPU): para lograr alto rendimiento en XPU en entornos Tensorflow
- Intel® Extension for Scikit-learn: para acelerar aplicaciones del ámbito de la biblioteca Scikit-learn
- Intel® Neural Compressor: para reducir los modelos de IA y acelerar el despliegue en CPUs y GPUs empleando técnicas de compresión de modelos, cuantización y pruning
- OpenVINO™ Toolkit: para facilitar el despliegue de modelos de IA en GPUs, CPUs y NPUs

En este lab nos vamos a centrar en la **distribución de Python optimizada: Intel Distribution for Python**

Instalación y setup de Intel Distribution for Python

Intel® Distribution for Python permite aprovechar todos los núcleos de CPU y las instrucciones más recientes para obtener un rendimiento escalable y cercano al nativo, especialmente en tareas numéricas y de aprendizaje automático. Gracias a extensiones como **dpnp**, es posible ejecutar código Python de forma acelerada en CPUs y GPUs sin necesidad de programar a bajo nivel. Estas herramientas están pensadas tanto para desarrolladores de IA, científicos de datos y expertos en HPC, como para estudiantes que desean aprender programación de alto rendimiento con Python. La plataforma integra bibliotecas optimizadas como oneMKL y herramientas de paralelismo como OpenMP

Para su instalación hay que descargar el paquete en Linux desde la URL y se obtendrá un fichero del tipo “**intelpython3-version-Linux-x86_64.sh**”. Para su instalación, seguir los siguientes pasos:

1. En una consola se ejecutar el script descargado `sh ./intelpython3-version.sh`
2. Presiona ENTER para aceptar las condiciones de la licencia
3. Selecciona la ruta de instalación o pulsar ENTER para instalarlo en la ruta por defecto

```
user@host% sh ./intelpython3-2025.1.0_196-Linux-x86_64.sh
```

```
Welcome to intelpython3 2025.1.0_196
```

```
In order to continue the installation process, please review the license agreement.
Please, press ENTER to continue
```

```
>>>
```

```
...
Do you accept the license terms? [yes|no]
>>> yes
...

intelpython3 will now be installed into this location:
/home/XXXX/intelpython3 >>>
```

Una vez instalado se puede usar la distribución de python de Intel cargando las variables de entorno con el comando `source env/vars.sh` desde la ruta donde se ha instalado por defecto. Si todo fue correcto el interprete de Python será **Python 3.12.8 | Intel Corporation:**

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ python3
Python 3.12.8 | Intel Corporation | (main, Feb 13 2025, 23:28:03) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

A continuación vamos a instalar el entorno de **Jupyter** en el nuevo interprete del Intel Distribution for Python con el comando `python3 -m pip install jupyter` y podremos arrancar el entorno de Jupyter usando el nuevo intérprete con `python3 -m jupyter notebook`

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ python3 -m pip install jupyter
....
user@host: ~/intelpython3$ python3 -m jupyter notebook
```

Ejercicios

Ejercicio 1: Vectorización en numpy

La explotación de la vectorización no es una teoría Es altamente recomendable utilizar librerías optimizadas en Intel oneAPI como NumPy, SciPy y otra

- Numpy explota el paralelismo inherente haciendo uso de librerías optimizadas en oneAPI como oneMKL
- Ej: 100x de aceleración en Numpy broadcasting respecto al código descrito como bucle
- Además el uso de numpy permite hacer uso de la memoria de forma eficiente

Uso de paralelismo

Para ver que se hace uso de la librería MKL de oneAPI podemos usar el comando de Numpy `np.show_config()`

```
import numpy as np
np.show_config()
....
SIMD Extensions:
  baseline:
    - SSE
    - SSE2
    - SSE3
    - SSSE3
    - SSE41
    - POPCNT
    - SSE42
  found:
    - AVX
    - F16C
    - FMA3
```

```
- AVX2
not found:
- AVX512F
- AVX512CD
- AVX512_SKX
- AVX512_CLX
- AVX512_CNL
- AVX512_ICL
```

En el cuaderno de Jupyter `numpy.ipynb` vamos a comprender:

- El uso de “ufuncs” es mucho más eficiente hacerlo directamente en **Numpy** que mediante bucles
- La indexación o desplazamientos de nuevo es más eficiente hacerla en **Numpy** que mediante bucles
- Las operaciones de agregación son más eficientes en **Numpy**
- Las operaciones de broadcast son más eficientes en **Numpy**
- Las operaciones condicionales son más eficiente en **Numpy**

Paquetes extra

Seguramente sea necesario instalar los paquetes de “pandas” y “matplotlib” para ejecutar los ejercicios del cuaderno:

```
user@host: ~/intelpython3$ source env/vars.sh
user@host: ~/intelpython3$ pip install pandas
user@host: ~/intelpython3$ pip install matplotlib
```

Ejercicio 2: Uso de numpy

En el cuaderno de Jupyter del ejercicio 2 denominado `numpy_ejercicio.ipynb` completar:

- Producto escalar de dos vectores eficientemente en Numpy
- Tabla de multiplicar filtrada aquellos números que sean múltiplos de 12 y múltiplos de 9.
- Black-Scholes es un modelo matemático que permite calcular el precio teórico de opciones financieras, en el cuaderno hay una implementación en bucle, se recomendando utilizar el formato “vectorial” de Numpy para acelerar los cálculos

Ejercicio 3: Uso de DPNP y el Data Parallel Extension en Python

El Data Parallel para Python es una extensión de Data Parallel para Python que amplían las capacidades numéricas de Python.

Permite utilizar la CPU o la GPU como un dispositivo de alto rendimiento donde acelerar los cálculos Para su uso hay varios paquetes básicos:

- **dpctl**: librería de control para seleccionar de dispositivos, la asignación de datos, datos tensoriales
- **dpnp**: extensiones paralelas para Numpy

El siguiente código muestra los dispositivos disponible en Python donde poder procesar los cálculos empleando la librería de control `dpctl`:

```
import dpctl
d = dpctl.select_gpu_device()
d.print_device_info()
```

Data Parallel Extensions

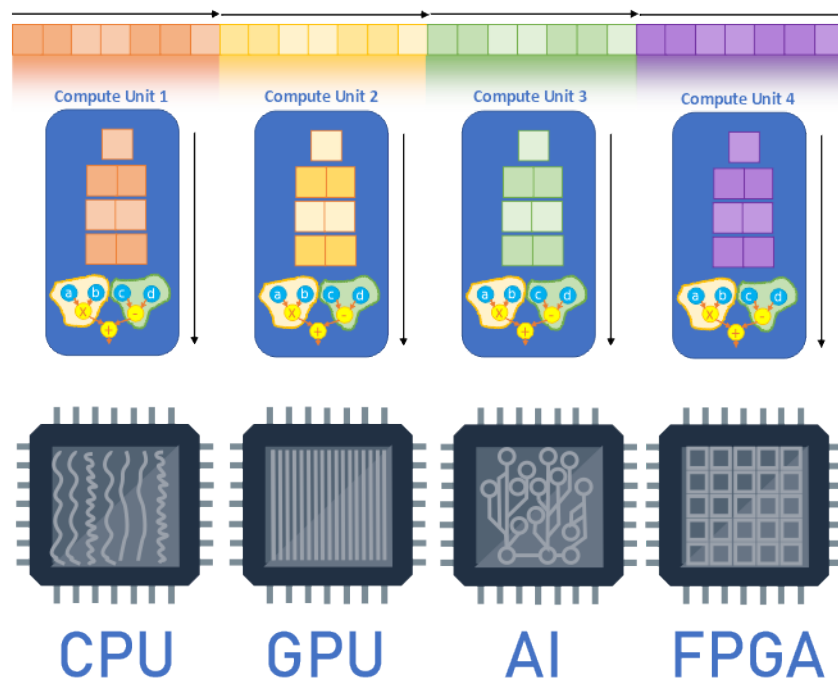


Figure 3: Fig: Data Parallel Extension para Python

```

user@host: ~/intelpython3$ python3 GPUdevice.py
Name           Intel(R) Graphics
Driver version  1.6.31294.120000
Vendor         Intel(R) Corporation
Filter string   level_zero:gpu:0

```

Además la librería `dpctl` permite almacenar datos en el dispositivo de forma sencilla con la clase `dpctl.tensor`:

```

from dpctl import tensor
import numpy
x_cpu = tensor.ones((2, 2), device="cpu")
x_gpu = tensor.ones((2, 2), device="gpu")
x_to_gpu = x_cpu.to_device("gpu")
x_np = numpy.ones((2,2))
x2_to_gpu = tensor.asarray(x_np, device="gpu")
w = tensor.asarray([x_cpu, x_gpu, x_np, x_to_gpu, x2_to_gpu], device="cpu")
y = w + w + 1
print("Tensor y device:", y.device)
print("Tensor y shape:", y.shape)
y_np = tensor.asnumpy(y)
print("max: ", numpy.max(y_np))

```

Desde el punto de vista del cómputo sigue el concepto Compute-follow-data que en lugar de mover grandes cantidades de datos, **el procesamiento se realiza allí donde los datos ya residen**.

Extensión de DataParallel para Numpy - dpnp

La librería `dpnp` (NumPy Drop-In Replacement for Intel® XPU) permite desarrollar aplicaciones en entorno *NumPy* en dispositivos paralelos. Básicamente permite seguir los conceptos expresados en el código desarrollado en *NumPy*.

Es decir, en un script Python escrito en *numpy*, donde una vez inicializado un array se multiplica cada componente por 4 que se procesa en la CPU:

```

# Original numpy code
import numpy
X = np.array([1,2,3])
Y = X * 4

```

se puede transformar fácilmente con la librería `dpnp` permitiendo realizar dichas operaciones en las XPU's (CPU o GPU) de forma sencilla:

```

# Transformed numpy into dpnp
import dpnp as np
X = np.array([1,2,3])
Y = X * 4
print("Array X allocated on the device:", X.device)
red = np.sum(Y)
print(red)

```

Contenido del laboratorio

En la carpeta “Ejercicio3” están dos cuadernos de Jupyter que permiten a alumno comprender estos conceptos:

- `get_started.ipynb` muestra el uso de la librería de control `dpctl`
- `dpep_scalability_journey_example-pairwise_distance.ipynb` muestra el procesado del calculo de la distancia entre puntos de tres componentes (ej. `nbody`) para 10x1024 elementos haciendo uso de Numpy, `dpnp` (en CPU y GPU)

Ejercicio 4

El cuaderno de Jupyter `ejercicio_dpnp.ipynb` muestra como poder utilizar la biblioteca DPNP para acelerar el cómputo en el XPU.

Ejemplo inicial

Inicialmente se muestra una multiplicación de matrices empleando el método `numpy.dot`, mostrando el código en Numpy y el equivalente en DPNP. Nótese que en la función `matrix_multiplication_dpnp` se selecciona inicialmente el dispositivo XPU que puede actuar como CPU o GPU mediante el método de la biblioteca de control: `select_cpu_device()/select_gpu_device()`. Posteriormente se envía desde el host los datos de a y b al acelerador (a_{xpu} , b_{xpu}) con la llamada de `np_dp.asarray(b, device=dev)` por último se calcula el producto matricial invocando a `np_dp.dot`:

```
def matrix_multiplication_dpnp(dev):  
  
    if (dev=='cpu'):  
        d = dpctl.select_cpu_device()  
    elif (dev=='gpu'):  
        d = dpctl.select_gpu_device()  
    d.print_device_info()  
  
    size = 8192  
    a = np.random.rand(size, size).astype(np.float32)  
    a_xpu = np_dp.asarray(a, device=dev)  
    b = np.random.rand(size, size).astype(np.float32)  
    b_xpu = np_dp.asarray(b, device=dev)  
  
    start_time = time.time()  
    c = np_dp.dot(a_xpu, b_xpu)  
  
    _ = c.asnumpy() #Ensure all computations are completed  
    end_time = time.time()  
  
    print(f"Matrix multiplication (DPNP on {dev}) completed.")  
    print(f"Time taken (DPNP on {dev}):", end_time - start_time, "seconds")  
    print("DPNP harnesses GPU acceleration for enhanced performance! Give it a try!")  
    return end_time - start_time
```

Ejemplo 1 (cálculo de π con el método de Montecarlo)

El método de Monte-Carlo es una técnica probabilística utilizada para estimar el valor de π generando puntos aleatorios dentro de un cuadrado y determinando cuántos caen dentro de un círculo inscrito.

Consideremos un círculo de radio $r = 1$ inscrito dentro de un cuadrado de lado $2r = 2$. La ecuación del círculo centrado en el origen es: $x^2 + y^2 \leq r^2$. Dado que el área del cuadrado es 4 y el área del círculo es π , la proporción de puntos dentro del círculo respecto al total de puntos generados aleatoriamente será aproximadamente: $\frac{\text{puntos dentro en el círculo}}{\text{puntos totales}} \approx \frac{\pi}{4}$

El alumno deberá de implementar la versión equivalente con DPNP, teniendo en cuenta que la inicialización aleatoria de “x” e “y” puede controlarse donde se alojan con el parámetro `device` de `dpnp.random.random` (más info en la documentación)

Ejemplo 2 (Kmeans)

El funcionamiento básico del algoritmo kmeans ya trabajado en la asignatura sigue los siguientes pasos:

La función `kmeans` implementa el algoritmo de K-means con los siguiente pasos:

1. **Inicialización:** Se seleccionan aleatoriamente K centroides iniciales (puntos representativos de cada grupo), esto se realiza con la variable `random_indices` que indexa a `centroids` en el código
2. **Asignación:** Cada punto del conjunto de datos se asigna al centroide más cercano (según distancia euclídea u otra métrica). En el código se calcula la distancia de cada punto a los centroides y se almacena en la variable `distances` y se le asigna el clúster correspondiente con la distancia menor con la función de `numpy` `np.argmin`
3. **Actualización:** Se recalculan los centroides como el promedio de los puntos asignados a cada grupo en la variable `new_centroids`
4. **Repetición:** Los pasos de asignación y actualización se repiten hasta que los centroides no cambian significativamente o se alcanza un número máximo de iteraciones. El “no cambio significativo” se implementa con el cómputo de la convergencia en la instrucción `shift = np.linalg.norm(centroids - new_centroids)`, mientras que la condición de parada por alcanzar el número máximo de iteraciones se implementa en el bucle `for iteration in range(max_iters):`

```
import numpy as np

def kmeans(X, k, max_iters=100, tol=1e-7, verbose=False):
    # X es un array (n_samples, n_features)
    n_samples, n_features = X.shape

    # Elegir k puntos iniciales aleatorios como centroides
    rng = np.random.default_rng(42)
    random_indices = rng.choice(n_samples, size=k, replace=False)
    centroids = X[random_indices]

    for iteration in range(max_iters):
        # Calcular distancias (n_samples x k)
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        # Asignar cada punto al centroide más cercano
        labels = np.argmin(distances, axis=1)

        # Calcular nuevos centroides
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # Verificar convergencia
        shift = np.linalg.norm(centroids - new_centroids)
        if verbose:
            print(f"Iter {iteration}, shift = {shift:.6f}")
        if shift < tol:
            break

        centroids = new_centroids

    return centroids, labels
```

ToDo: implementación con DPNP El alumno desarrollará la implementación con DPNP. Para ello se proporciona un esqueleto donde ya se refleja el movimiento del dataset original al procesador XPU mediante la operación `X_dnp = np.asarray(X_np, device="cpu")`. Posteriormente para mostrar el resultado de los nuevos centroides, al estar almacenados en la XPU, hay que moverlos a la memoria de “host” para poder ser visualizados pudiendo hacerse con el comando `dnp.asarray()`. Además de estos aspectos se recomienda al estudiantado las siguientes recomendaciones:

- Elegir k puntos iniciales aleatorios como centroides, pero no existe una correspondencia en la librería DPNP de la función `rng.choice` por lo que el cálculo de `rng` y `random_indices` se deben realizar como en *Numpy* original, y luego enviar la variable `random_indices` al dispositivo XPU con el comando `dnp.asarray`. La indexación posterior al hacerse en el dispositivo con datos que están en el dispositivo es correcta

- La verificación de la convergencia, hay que tener en cuenta que la variable `shift` que calcula la norma de los centroides “antiguos” respecto a los “nuevos”, se efectúa en la XPU por lo que la variable “shift” estará almacenada en el dispositivo, pero la condicional `if shift < tol` se procesa en el host no pudiendo comparar ambas variables al estar en ámbitos distintos (XPU para `shift` vs `tol` en el host). En este caso se recomienda traer el contenido de `shift` al host para procesarlo de forma coherente. Esto es posible con la función `dpnp.asnumpy` que convierte una variable alojada en el XPU a numpy.

```
import numpy as npy
import dpnp as np

def kmeans_dpnp(X, k, max_iters=100, tol=1e-7, verbose=False):
    # X es un array (n_samples, n_features)
    n_samples, n_features = X.shape

    # Elegir k puntos iniciales aleatorios como centroides
    #.....

    return centroids, labels

X_dpnp = np.asarray(X_np, device="gpu") # Convert from numpy to dpnp array

start = time.time()
centroids, labels = kmeans_dpnp(X_dpnp, k=5, verbose=False)
print("Time to Kmeans with DPNP:", time.time() - start)
print("Centroids:\n", centroids.asnumpy())
```

Resultados esperado

Al finalizar esta práctica, el alumno será capaz de:

- Comprender explotar paralelismo SIMD y multi-hilo en Python de forma transparente mediante la biblioteca de Numpy
- Comprender el funcionamiento básico de la extensión de Intel® DPNP y cómo se integra con la explotación de aceleradores